

Article

Column-Wise Autoencoder Representation Learning for Intrusion Detection in Multi-MEC Edge Networks

Min-Gyu Kim  and Jonghyun Kim * 

Department of Computer and Information Security, Sejong University, Seoul 05006, Republic of Korea; skystarloid@sju.ac.kr

* Correspondence: jhk@sejong.ac.kr

Abstract

Mobile Edge Computing (MEC) is a key enabler of 5G/6G services, but multi-base-station deployment enlarges the attack surface and motivates edge-native intrusion detection systems (IDSs). Existing MEC-based IDSs are mainly single-node or centralized, which struggle with heterogeneous traffic across next-generation Node Bs (gNBs) and incur latency and network load due to data aggregation. To address these limitations, this paper proposes a Column-Wise Autoencoder Ensemble (CW-AE) distributed learning framework for multi-MEC environments. Each MEC node trains column-wise autoencoder encoders locally to extract compact latent features, and a master MEC trains a stacking-based meta-classifier using concatenated latent features, avoiding raw traffic transfer and parameter averaging. By preserving node-specific behavior while integrating heterogeneous features, CW-AE improves detection performance and reduces communication overhead. Using the real-world 5G-NIDD dataset collected from two physical 5G base stations, we compare local single-node, centralized, and CW-AE-based distributed learning. The results show that CW-AE achieves superior detection capability and network efficiency, making it suitable for scalable edge IDS deployments.

Keywords: network security; Mobile Edge Computing; intrusion detection system; distributed learning; stacking ensemble

1. Introduction

5G has been commercially deployed for about six years, and research toward 6G commercialization around 2030 is actively underway. As related technologies have matured, a wide range of services built on 5G infrastructure has emerged, increasing the importance of Mobile Edge Computing (MEC). The network closest to the User Equipment (UE) is the Radio Access Network (RAN). Since MEC is deployed in the RAN and provides computing services there, it offers advantages such as achieving lower latency and improved energy efficiency compared to communicating with remote service servers.

6G aims for even higher communication speeds and lower latency compared to 5G, and has added target metrics for new service areas such as intelligence, security, and ubiquitous connectivity. The 6G use case scenario services defined in the 6G framework recommendation published by International Telecommunication Union Radiocommunication Sector (ITU-R) [1] are as follows:

1. Immersive communication: Services such as extended reality (XR), holographic communication, and mixed video/audio traffic.



Academic Editor: Christos Bouras

Received: 15 February 2026

Revised: 12 March 2026

Accepted: 20 March 2026

Published: 21 March 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

2. Hyper Reliable and Low-Latency Communication: Use cases including smart industries, automated processes, energy services, and remote medical treatment.
3. Massive communication: Services supporting smart cities, transportation, logistics centers, healthcare, energy, and agriculture.
4. Integrated AI and communication: Applications such as autonomous driving, digital twins, medical assistance, and robotics.
5. Integrated sensing and communication: 6G-based navigation, motion and gesture detection, environmental monitoring, and provision of sensing information for AI, XR, and digital twin applications.
6. Ubiquitous connectivity: Ranging from Internet of Things (IoT) services to basic broadband access.

As shown above, it is expected that the introduction of MEC will become increasingly effective for services that need to meet these target requirements or that leverage artificial intelligence (AI).

In particular, research on deploying AI models within mobile networks for use in services or security has continued steadily to this day. Most studies on AI models for detecting attack traffic or anomalies have been conducted under the assumption that training takes place on a single node. In today's advanced 5G and 6G environments, the importance of edge networks has increased, leading to the deployment of multiple MEC nodes. As a result, each MEC can become a target for attacks, and attack traffic can be distributed across multiple nodes. Therefore, distributed learning across multiple nodes within the network needs to be discussed [2–6].

As mobile networks evolve to connect an ever-growing number of devices, the amount of data generated also increases significantly. Since such data is produced in a distributed manner across different base stations, collecting and processing it entirely on a central server inevitably causes additional latency. As AI models become increasingly complex, distributed learning across multiple MEC nodes becomes essential for efficient training while maintaining low latency [7,8].

Our main novelty lies in the learning procedure: we design an intrusion detection AI model using a stacking ensemble structure that trains across multiple MECs while keeping node-specific traffic and avoiding heavy central aggregation. Specifically, to reduce the burden on Worker and Master nodes in a distributed learning environment, we partition the features of the original training data in a column-wise manner to compress the data, enabling the meta-classifier to learn patterns from diverse nodes. Each node trains an encoder on its own feature subset, and only compact latent features are sent to the master for meta-classifier training, avoiding repeated parameter averaging used in federated learning.

In this paper, we propose a distributed learning approach for multi-MEC environments. The proposed method considers minimizing network traffic overhead while enabling the sharing of training data within mobile networks. To evaluate the effectiveness of the proposed approach, we utilized the 5G-NIDD dataset, which contains normal and denial-of-service (DoS) traffic collected from two real base stations. The main contributions of this paper are as follows:

- We propose a novel distributed learning framework for multi-MEC environments, where base models are trained locally at each MEC node and a meta-classifier is aggregated at the master MEC node. This design reflects realistic mobile network conditions and ensures scalability.
- Through extensive experiments using the 5G-NIDD dataset, we demonstrate that the proposed distributed learning approach achieves performance comparable to centralized learning, with slight gains in Accuracy, Recall, and F1-score, while maintaining near-perfect AUC performance.

- While centralized learning used three encoders as base models for meta-classifier training, the proposed distributed learning approach combined latent features from six encoders—three base encoders from each node (MEC1 and MEC2). Despite doubling the input dimension to the meta-classifier from 21 to 42, the proposed method achieved comparable or slightly improved performance metrics. It absorbed data variance between next-generation Node Bs (gNBs), securing more stable integrated performance. This represents a significant achievement, overcoming the limitations of existing stacking-based IDS research where increased input dimensions typically lead to higher model complexity and learning difficulty. It simultaneously achieves model scalability and performance enhancement.

The remainder of this paper is organized as follows: Section 2 introduces background concepts on autoencoders and stacking ensemble models; Section 3 reviews related work in distributed learning and network security; Section 4 presents the proposed multi-MEC-based stacking ensemble distributed learning framework; Section 5 describes experiments across four stages (local single-node, centralized, multi-edge distributed, and logical N-node scalability analysis); Section 6 analyzes experimental results; and Section 7 concludes with contributions and future directions.

2. Background

2.1. Autoencoder

An autoencoder is a representative model of unsupervised learning. Its general structure is illustrated in Figure 1. The objective of an autoencoder is to train the model so that the output data closely resembles the original input data.

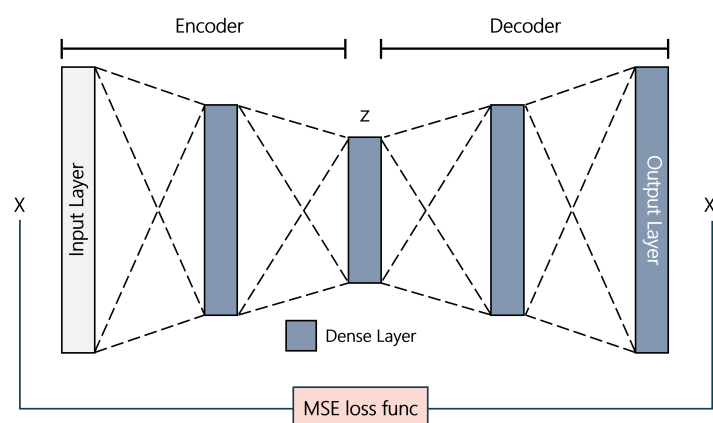


Figure 1. Autoencoder architecture.

To achieve this, the model is composed of two parts: the encoder and the decoder. The encoder is designed with progressively fewer nodes in each layer. This reduces the dimensionality of the input data and performs a compression process. The output of the encoder is therefore referred to as a 'latent variable' or 'code'. Conversely, the decoder is designed with progressively more nodes in each layer. The final layer of the decoder has the same dimensionality as the input layer. In this way, the decoder expands the latent variable produced by the encoder back to the size of the original input data. The output of the decoder is then compared to the input data using the Mean Squared Error (MSE) loss function.

Because the autoencoder is trained to minimize the MSE value, the encoder is optimized to effectively compress the input data into a latent variable, while the decoder is optimized to reconstruct the input data as closely as possible from the latent variable.

Given these characteristics, autoencoders are mainly applied in two ways:

1. **Dimensionality reduction:** When training data has very high dimensionality, only the encoder part of a trained autoencoder can be used to reduce the number of dimensions, thereby lowering the training cost.
2. **Denoising:** Since the autoencoder is trained to produce outputs that are as close as possible to the original input, it can be applied to remove noise from the original data (e.g., images).

In this study, we leverage the encoder's dimensionality-reduction capability to obtain compact latent representations with lower computational overhead.

2.2. Stacking Ensemble Model

The ensemble technique is a method of constructing an overall model by combining multiple individual models. Ensemble methods can be categorized into voting, bagging, boosting, and stacking. Among these, the stacking ensemble method consists of multiple base models and a meta-classifier, as illustrated in Figure 2. The training process proceeds as follows:

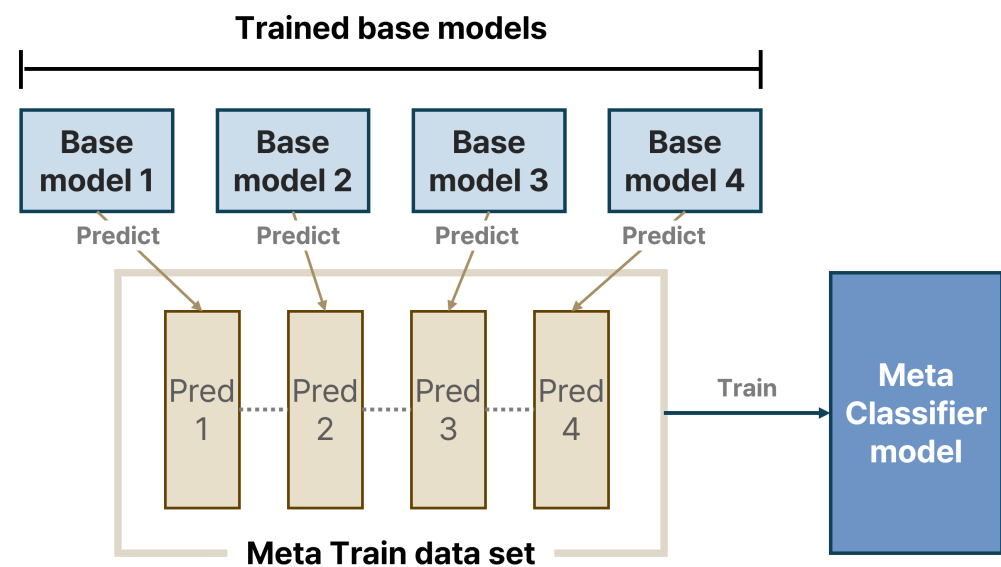


Figure 2. Stacking ensemble model training process.

1. Train the base models using the training dataset.
2. Use the trained base models to make predictions on the validation set. The predictions generated here are outputs on data not used for training the base models, preventing information leakage.
3. Concatenate the predictions generated by each base model on the validation set to create the meta train dataset, which is the training data for the meta-classifier model.
4. Train the meta-classifier model using the meta train dataset and the corresponding validation set's ground truth labels.

The core principle of stacking is that the meta-classifier must use predictions generated on data not used for training the base models as input. If predictions generated on the train set used for training the base models are used to train the meta-classifier, the meta-classifier will learn the base model's training error patterns and fitted bias prediction characteristics specialized to the base models' training data. This leads to double overfitting, where the meta-classifier repeats the same overfitting, resulting in significantly degraded generalization performance on unseen data.

The predict procedure follows the same data flow as the train procedure. Data to be predicted is input to all base models. The predict results from each base model are

concatenated and then input to the meta-classifier, which outputs the final predict result. The main idea of stacking is to train base models first and then train a meta-classifier on their outputs. The strengths are better generalization and less overfitting, as the meta-classifier learns to combine the strengths of multiple base models.

Based on the procedure described above, this paper designed a stacking ensemble model composed of multiple encoders as base models.

3. Related Work

3.1. Centralized AI-Based Intrusion Detection in 5G/6G Networks

This subsection briefly introduces centralized IDS studies that collect data at the core/cloud and shows why this setting is less suitable for MEC. Since the commercialization of 5G, various studies have been conducted on AI-based intrusion detection systems (IDSs) designed for deployment in the core network or cloud environments. Early studies adopted a centralized approach. In 5G and 6G environments, centralized AI-based IDSs have long been the mainstream approach, as they can leverage abundant computational resources and integrated data accessibility to achieve high detection performance.

Autoencoder- and deep neural network (DNN)-based architectures have been widely used, particularly for Internet of Things (IoT) and mobile traffic analysis. Yadav et al. and Tu et al. implemented high-precision detection at the central server using an autoencoder (AE)-DNN hybrid model and a Pseudo-Siamese Stacked Autoencoder (PSSAE), respectively [4,5]. These methods achieved high accuracy and detection rates on datasets such as UNSW-NB15 and KDDTest+/-21, demonstrating the effectiveness of representation learning-based IDS. However, their evaluations primarily relied on outdated datasets, and validation using real 5G traffic was limited.

Meanwhile, research on high-performance IDS combining convolutional neural network (CNN)-based feature extraction with deep learning models has also been actively conducted. Loukas et al. and Nithyalakshmi et al. proposed hybrid IDS models combining CNN-mixture of experts (MoE) structures and clustering/association rule-based feature selection with DNNs, achieving high F1-scores and low false alarm rates across diverse environments including 5G-NIDD, NSL-KDD, and CICIDS2017 [2,3]. However, the increasing complexity of these models raises concerns regarding computational overhead and communication load when applied to MEC-based distributed architectures.

Furthermore, within centralized structures, attempts have been made to improve early detection of flow-based traffic. Djaidja et al. proposed an early flow classification method combining long short-term memory (LSTM) and gated recurrent unit (GRU) mechanisms with attention to detect attacks before flow termination. Experiments using the CICIDS2017 and 5G-NIDD datasets demonstrated that the proposed model effectively reduced detection latency and the number of packets required for early detection [6]. However, since the evaluation primarily focused on accuracy, the study lacked analysis of false positive rate variation in early detection scenarios. These centralized approaches differ in feature extraction and evaluation focus: AE/DNN- and CNN-based models emphasize detection accuracy across datasets, whereas early-flow methods prioritize detection latency and packet efficiency.

Although these centralized IDS studies provide high detection performance, they also face limitations such as increased latency, network traffic load, and unsuitability for distributed environments due to model scaling and data aggregation. Consequently, their applicability to MEC-based distributed architectures in 5G and 6G networks remains limited.

3.2. Distributed and Edge-Based Intrusion Detection Systems (MEC Environments)

This subsection reviews IDS studies that operate across MEC/edge nodes and share models or features instead of raw traffic. Distributed resource management in MEC has been widely studied in wireless-powered and heterogeneous MEC settings. Yun et al. proposed a distributed DRL scheme for 5G multi-RAT URLLC/eMBB task offloading with MEC resource allocation, while Zhang et al. presented a fully decentralized framework for secure data offloading and resource allocation against hybrid intrusions in IIoT [9,10]. In WP-MEC, Liu et al. proposed a two-stage multi-agent DRL framework (TMADO) for multi-HAP networks that jointly optimizes HAP transmit power, WPT duration, and WD offloading decisions/CPU frequency under delay and demand constraints [11]. These works highlight decentralized decision making and resource-aware design, but focus on computation/energy efficiency or secure offloading rather than IDS learning, which motivates IDS-specific distributed learning. Similar to these studies, distributed IDSs should also consider computational and energy efficiency, as demonstrated in our experimental results where distributed learning reduces training time and distributes computation across nodes. MEC-enabled 5G AIoT networks face attack detection and mitigation challenges, reinforcing the need for edge-native protection [12]. In edge-based networks, various distributed detection studies have been proposed to address the latency and large-scale traffic transmission overhead inherent in centralized IDSs. Man et al. and Uddin et al. proposed architectures for efficient edge-level detection in IoT and Internet of Vehicles (IoV) environments, respectively, achieving reduced latency and improved resource efficiency through techniques such as federated learning and feature selection [13,14].

In addition, studies based on Split Learning have proposed various optimization methods to reduce communication costs and enhance privacy protection. Lyu et al. and Oh et al. presented U-shaped SL structures and SplitFC, which combines adaptive dropout and quantization, demonstrating architectures that significantly reduce communication volume while maintaining accuracy [15,16]. Additional federated IDS studies also explore clustered FL and BERT-based FL to handle heterogeneity and improve edge efficiency [17,18]. Other FL-based IDS works include privacy-preserving FL, knowledge-distillation FL, in-network FL, and stacked-unsupervised FL, which offer different ways to balance privacy, accuracy, and communication [19–22].

Meanwhile, Nakip et al. proposed a G-Network-based online federated learning IDS (DOF-ID) for multi-component environments such as supply chains. By enabling each node to perform both local and collaborative learning, the proposed framework achieved consistent improvements in detection performance on the Kitsune and Bot-IoT datasets [23]. Overall, federated approaches emphasize privacy-preserving parameter sharing, split-learning variants focus on communication reduction through model partitioning, and online FL schemes (e.g., DOF-ID) target sustained performance under distributed and multi-component settings.

However, existing distributed IDSs still face fundamental challenges, including communication overhead, model consistency across nodes, and handling of heterogeneous distributed data. Therefore, a new distributed learning approach is required to ensure both efficiency and detection performance in MEC-based multi-node architectures.

3.3. Ensemble and Stacking-Based Models for Intrusion Detection

This subsection summarizes ensemble/stacking IDS studies and clarifies their limits when applied to MEC settings. Ensemble and stacking-based IDS research has advanced toward combining the strengths of multiple models to more precisely learn complex network traffic patterns. Stacked deep learning has also been applied to edge-based intelligent threat detection in IoT networks [24]. In particular, autoencoder-based stacking architectures offer

advantages in effectively capturing latent characteristics of abnormal traffic. Cui et al. and Fathima et al. proposed an imbalance-aware approach that integrates stacked autoencoder (SAE)-based feature extraction with GMM-WGAN, and a two-stage hybrid model composed of stacked autoencoders and a random forest classifier, respectively. Their methods demonstrated strong performance in minority-class and sparse attack detection [25,26].

Hybrid ensemble techniques combining traditional machine learning and deep learning have also been explored. Abbas et al. combined Random Forest with Recursive Feature Elimination (RF-RFE), achieving both high accuracy (98–99%) and low computational cost on datasets such as NSL-KDD, UNSW-NB15, and CSE-CIC-IDS2018 [27]. Yin et al. integrated CNN and bidirectional LSTM (Bi-LSTM) into a Multi-Scale Arbitration Dense Network (MSCBL-ADN), improving both accuracy and time efficiency for low-rate distributed denial-of-service (LDDoS) attack detection and demonstrating stable gains over state-of-the-art methods [28]. These ensemble variants illustrate different trade-offs. Stacked AE-based models focus on latent features for imbalance and rare attacks, while hybrid ensembles prioritize accuracy with reduced computational cost. Our proposed approach combines the advantages of stacked AE-based models by using column-wise autoencoder encoders to extract compact latent features, while maintaining computational efficiency through distributed training across multiple MEC nodes.

In addition, new efforts have combined dataset construction with stacking-based ensemble architectures to address the limitations of anomaly-based network intrusion detection systems (NIDSs). Ali et al. and Rais et al. introduced the CIPMAIDS2023-1 dataset and applied stacking-based ensemble models, reporting improved performance in a GNS-3 environment and achieving an F1-score of 98.24% [29,30].

Although existing ensemble-based IDS approaches secure high accuracy and generalization capability through diverse model combinations, they still face structural limitations such as increased training cost, difficulty in real-time deployment, and greater communication overhead in distributed environments. Therefore, in MEC-based networks characterized by resource constraints and heterogeneous traffic patterns, a more lightweight and distributed-learning-friendly stacking model design is required.

In this study, we aim to address these limitations of existing centralized and distributed environments by reducing communication overhead and applying an ensemble model capable of training a more powerful AI model under distributed settings. To provide a structured comparison, Table 1 summarizes representative centralized, distributed, and ensemble-based IDS studies and contrasts them with our CW-AE approach.

Table 1. Summary of representative IDS studies and comparison with the proposed approach.

Category	Representative Studies (Examples)	Learning Setting	Key Limitation/Gap
Centralized IDS	AE/DNN, CNN-hybrid, early-flow models [2–6]	Centralized training at core/cloud	High aggregation cost, limited MEC suitability
Distributed/edge IDS	Federated, split learning, online FL, privacy-preserving FL, knowledge-distillation FL, in-network FL, stacked-unsupervised FL [13–23]	Model/gradient sharing across nodes	Communication overhead, heterogeneous data handling
Ensemble/stacking IDS	Stacked AE, hybrid ensembles, stacking-based ensemble [24–30]	Ensemble aggregation at a single node	Scaling to MEC is non-trivial, higher training cost
This work (CW-AE)	Column-wise AE encoders + stacking meta-classifier	Local representation learning + master aggregation	Mitigates raw-data transfer and preserves node-specific behavior

4. Proposed Method

For clarity, we provide a short autoencoder overview in Section 2. Here, we summarize its role in our proposed framework: each encoder compresses its assigned feature subset into a compact latent feature, and these latent features are used by the stacking meta-classifier for final classification.

This study proposes a distributed learning method across edge nodes using a stacking ensemble technique, with the goal of minimizing data transfer between nodes.

As illustrated in Figure 3, multiple gNBs are deployed across different regions, and MEC nodes provide access to specific 5G services. The proposed approach is applicable in such an environment. In this context, an attacker may target an MEC that delivers 5G edge services, attempting to launch attacks through any connected gNB.

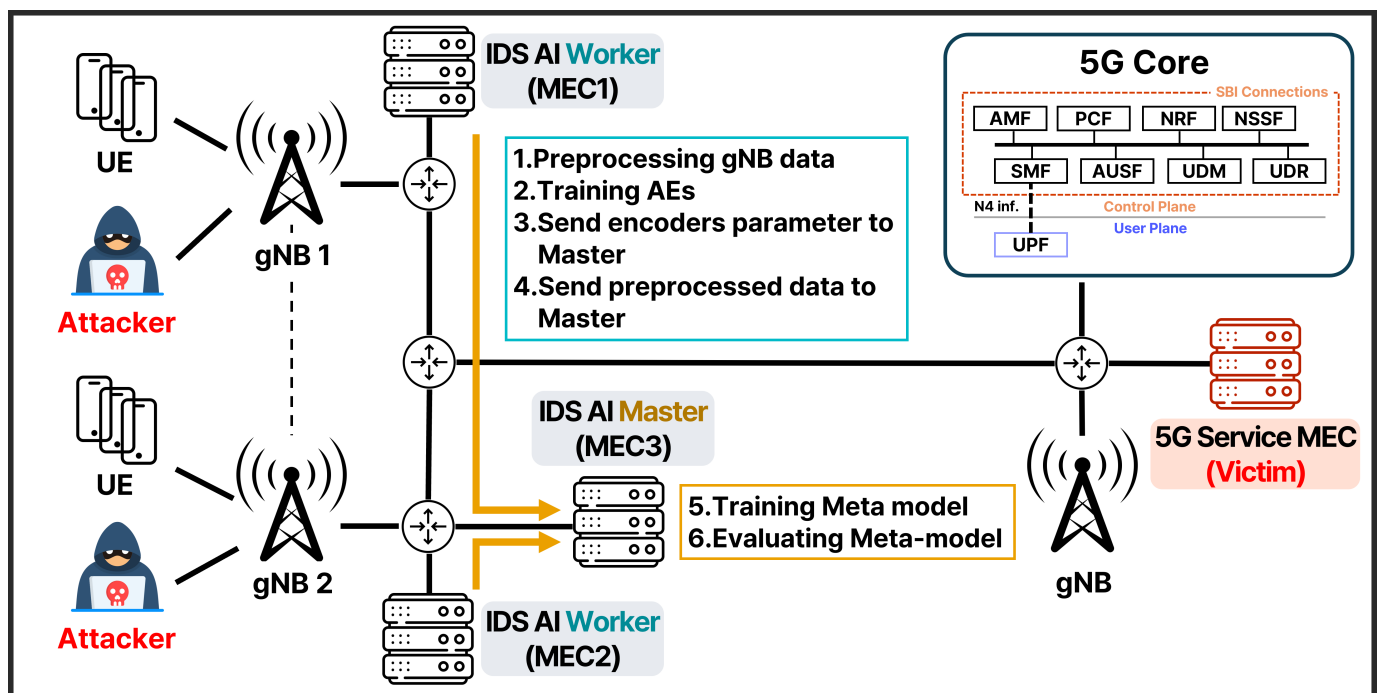


Figure 3. Proposed distributed learning architecture in multi-MEC edge network.

From the administrator's perspective, attacks can be detected either within the 5G core network or in the RAN network. Detection in the RAN network can be performed directly within the base stations or by deploying additional intrusion detection systems (IDS) in the RAN domain.

In this paper, we propose a training approach for AI-based IDS models that are distributed across MEC nodes in the RAN, rather than adopting a centralized IDS in the core network. A centralized approach requires aggregating all traffic into the core, which leads to resource consumption and increased latency. In contrast, since attacks targeting MEC nodes often bypass the core network, it is more effective to implement detection directly at the MEC level, where attacks are likely to occur. Accordingly, this study adopts a distributed IDS architecture based on MEC and designs a training procedure to support it.

The distributed learning procedure across multiple edge nodes proposed in this paper is as follows:

1. Each MEC located near a base station (Worker MEC1, MEC2) holds the base models of the stacking ensemble, while MEC3 (Master) holds the meta-classifier.
2. Each Worker MEC collects data from its associated base station. The data is split into training sets for the base models and for the meta-classifier in a 7:3 ratio, and the base models are trained.

3. Each worker node (MEC1, MEC2) preprocesses the original traffic data stored in CSV format (e.g., scaling, missing value handling), then converts it into a pickle file containing both the preprocessed data and the parameters of the trained base model. This pickle file is transmitted to the master node (MEC3).
4. The Master MEC (MEC3) integrates the received base model parameters to construct the full ensemble model.
5. Finally, the master node (MEC3) uses the meta-classifier training data, derived from portions of the datasets of MEC1 and MEC2, to train and test the complete ensemble model and evaluate its performance.

Cooperation among MECs follows a star layout. Encoders are trained at worker MECs, and only encoder parameters and preprocessed validation/test splits are sent to the master MEC. Raw packets and gradients are not exchanged between workers, and there is no direct worker-to-worker synchronization, which lowers coordination cost. Unlike federated learning, we do not repeatedly average parameters or gradients: each worker sends encoder parameters once, and the master trains the meta-classifier on concatenated latent features from all workers, preserving local patterns without repeated synchronization. This architecture reduces communication by sending only encoder parameters and compact latent features, keeps node-specific patterns through separate encoders, and avoids mismatch from global averaging because the meta-classifier learns on concatenated features.

The key design choice of the proposed framework is to separate local representation learning from global decision learning. Instead of aggregating raw traffic or repeatedly synchronizing model parameters across MEC nodes, each worker learns a compact latent representation only from its own feature subset. This column-wise design reduces the dimensionality handled by each local encoder, preserves node-specific traffic characteristics, and enables the master node to build a global detector from concatenated latent features rather than from centrally aggregated raw data. As a result, the proposed framework is better suited to heterogeneous multi-MEC environments, where direct data aggregation may blur local traffic patterns and increase communication overhead.

The structure of the stacking ensemble model proposed in this study for distributed learning is illustrated in Figure 4. The base models are constructed using the encoder components of autoencoders, while the meta-classifier is implemented with an XGBoost classifier. Each autoencoder encoder consists of two fully connected layers that compress the input features: the first layer reduces the input dimension to 16, and the second layer further reduces it to 12, finally producing a latent representation of dimension 7.

More specifically, the input features are partitioned into fixed column groups, and each encoder processes only its assigned subset to produce a low-dimensional latent vector. This design enables each base model to specialize in a limited feature space instead of learning from the full input dimension. The resulting latent vectors are then concatenated to form the input of the meta-classifier, which learns the final decision boundary for intrusion detection. Because only compact latent representations are propagated to the meta level, the proposed architecture reduces the dimensionality of the data handled in the final stage and lowers the communication burden compared with transferring raw features. At the same time, the use of separate encoders allows each feature subset to preserve its own local structure, while the XGBoost meta-classifier captures cross-subset interactions from the concatenated latent space. In this way, the model combines lightweight local encoding with expressive global classification, making it suitable for distributed training across multiple MEC nodes.

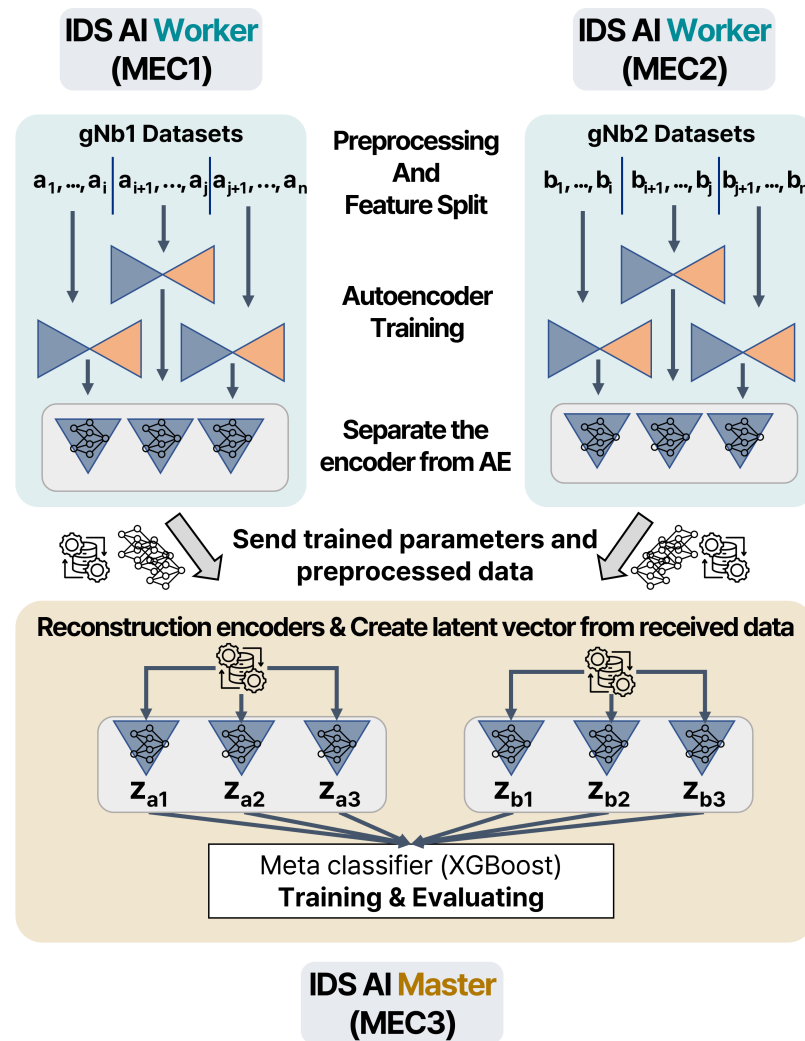


Figure 4. Proposed distributed stacking ensemble model procedure in multi-MEC edge network.

A key characteristic of this model is that it sequentially partitions the features of the training data across multiple encoders, which locally compress the data to form the input dataset for the meta-classifier. This design addresses the limitations mentioned in Related Work: (1) it reduces communication overhead by transmitting only compact latent features instead of raw data, addressing the high aggregation cost of centralized IDS; (2) it handles heterogeneous data by preserving node-specific patterns through separate encoders at each node, addressing the heterogeneous data handling challenge in distributed IDS; and (3) it enables efficient scaling to MEC by distributing computation across nodes and reducing training cost through column-wise feature partitioning, addressing the scaling and training cost issues in ensemble IDS.

The training procedure is as follows:

1. Split the columns of the training dataset evenly, according to the number of autoencoders used as base models.
2. Train each autoencoder using its assigned subset of the training data as input.
3. Extract the trained encoders from the autoencoders and utilize them as the base models of the stacking ensemble.
4. Feed the training data into the base encoders, concatenate the resulting latent features, and construct the dataset for the meta-classifier.
5. Train the XGBoost meta-classifier using the dataset composed of the latent features.

Algorithm 1 summarizes the overall learning procedure of the proposed architecture, which corresponds to the conceptual workflow illustrated in Figure 4. The algorithm proceeds in four major stages as follows.

Algorithm 1 Distributed stacking ensemble training workflow across multi-MEC nodes

Require: Local datasets D_1 (MEC1), D_2 (MEC2)

Require: At MEC3: identical encoder architecture and received encoder parameters (PyTorch `state_dict`)

Require: Feature-partition vector $\mathbf{s} = [s_1, s_2, s_3]$ splitting the input into three fixed subsets assigned to three autoencoders

Ensure: Meta classifier f_{meta} and threshold τ^*

AE_{ij}: *j*-th autoencoder at worker *i*

Enc_{ij}: encoder reconstructed at MEC3 using parameters θ_{ij}

Concat($\cdot$): feature-wise concatenation

Worker nodes (MEC1, MEC2)

for $i \in \{1, 2\}$ **do**

 Load and preprocess D_i into (X_i, y_i)

 Split into train/val/test sets

 Slice X_i^{train} into 3 subsets using $\mathbf{s}$

for $j = 1$ to 3 **do**

 Train AE_{ij} using subset j

 Extract parameters $\theta_{ij} = AE_{ij}.\text{encoder.state_dict}()$

 Save θ_{ij} via `torch.save()`

end for

 Transmit $\{\theta_{i1}, \theta_{i2}, \theta_{i3}\}$ and $\{X_i^{\text{val}}, y_i^{\text{val}}, X_i^{\text{test}}, y_i^{\text{test}}\}$ to MEC3

end for

Master node (MEC3)

Load all θ_{ij} and reconstruct Enc_{ij}

Merge validation/test sets:

$X^{\text{val}} = \text{vstack}(X_1^{\text{val}}, X_2^{\text{val}})$, $X^{\text{test}} = \text{vstack}(X_1^{\text{test}}, X_2^{\text{test}})$

$y^{\text{val}}, y^{\text{test}}$ in the same way

Set $(X^{\text{train}}, y^{\text{train}}) \leftarrow (X^{\text{val}}, y^{\text{val}})$

Meta dataset construction

for $* \in \{\text{train, val, test}\}$ **do**

for $i \in \{1, 2\}$ **do**

 Slice X_i^* into 3 subsets according to $\mathbf{s}$

 Create latent vectors $z_{ij}^* = Enc_{ij}(X_{ij}^*)$ for $j = 1, 2, 3$

 Construct node-specific meta dataset:

$Z_i^* = \text{Concat}(z_{i1}^*, z_{i2}^*, z_{i3}^*)$

end for

 Merge node-level meta datasets:

$Z^* = \text{vstack}(Z_1^*, Z_2^*)$

 Convert labels to binary form: $\tilde{y}_i^* = 1(\text{if } y_i^* \neq 0)$

end for

Train f_{meta} with $(Z^{\text{train}}, \tilde{y}^{\text{train}})$

Threshold selection

Sweep $t \in [0, 1]$ with step 0.01 and compute F1-score with $(Z^{\text{val}}, \tilde{y}^{\text{val}})$

Select τ^* that maximizes validation F1-score

Final evaluation

Evaluate f_{meta} with $Z^{\text{test}}, \tilde{y}^{\text{test}}$ and threshold τ^*

Report Accuracy, Precision, Recall, F1-score, AUC

5. Experiments

In this study, we considered four experimental scenarios to compare AI training methods in multi-node environments: (1) local single-node model training, (2) centralized model training, (3) the proposed 2-node distributed model training, and (4) logical N-node distributed model training for scalability evaluation. This section describes the experimental settings for each scenario and compares both the traffic generated during training and the attack detection performance of the models.

To clarify the evaluation scope, this comparison is designed as a system-level scenario comparison rather than a strictly capacity-controlled algorithm-only comparison. The three scenarios share the same proposed model structure (column-wise autoencoder with stacking ensemble), but the number of integrated encoders and the meta-input di-

dimensionality vary by deployment scenario. In the centralized setting, the meta-classifier uses three encoders trained on the aggregated data, resulting in a meta-input dimension of 21 (3 encoders $\times$ 7 latent dimensions). In the multi-node distributed setting, the master-side ensemble integrates latent features from two MEC nodes, resulting in six encoder outputs and a meta-input dimension of 42 (6 encoders $\times$ 7 latent dimensions). Accordingly, the local vs. multi-node performance difference is interpreted by jointly considering deployment-scenario effects and representational integration, rather than being treated as a pure algorithm-only superiority claim under identical input dimensionality. For fairness between centralized and distributed settings, we use identical encoder architectures and training configurations: feature split (8,8,9), three encoders per node (centralized uses three encoders, distributed uses six encoders from two nodes), XGBoost meta-classifier, and 20 training epochs for base models. All experiments use a fixed random seed of 42 for reproducibility.

5.1. Experimental Setup

5.1.1. Experimental Environment Configuration

The experimental environment was configured to simulate a system with three MEC nodes, as shown in Figure 5. All nodes were workstations running Windows 11 OS, and Docker Desktop (Docker version 28.3.2) was employed to build containerized environments to ensure consistent execution conditions across nodes. The PyTorch 2.8.0 deep learning framework was used for model implementation.

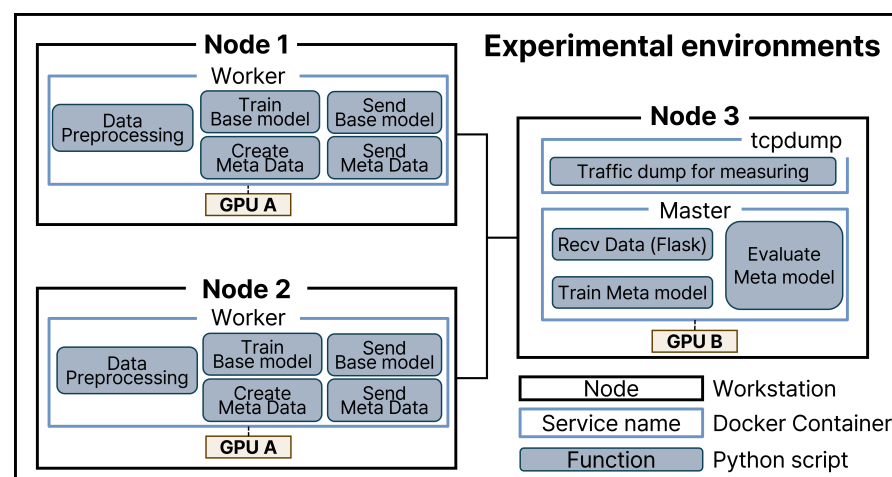


Figure 5. Experimental environments

Each node contained either a worker or a master service container, and additionally included a TCPDUMP service container (tcpdump version 4.9.0) for measuring traffic performance. The worker and master containers provided functionalities for model training and data transmission and reception. To enable GPU-based model training within Docker containers, container images developed by NVIDIA were used [31]. Furthermore, to capture traffic in conjunction with the training containers, the tcpdump container developed by Kaazing [32] was employed.

Node 1 and Node 2 were equipped with identical hardware environments, each consisting of an AMD Ryzen7 7800X3D CPU (Advanced Micro Devices, Santa Clara, CA, USA), 128 GB of memory, and an NVIDIA GeForce RTX 5070 Ti GPU (NVIDIA Corporation, Santa Clara, CA, USA).

In contrast, Node 3, which served as the Master node, was configured with a higher-performance AMD Ryzen9 9950X CPU (Advanced Micro Devices, Santa Clara, CA, USA),

128 GB of memory, and an NVIDIA GeForce RTX 5090 GPU (NVIDIA Corporation, Santa Clara, CA, USA) to handle computation and model integration tasks.

For the logical N-node distributed learning experiments, we extended the experimental setup to evaluate the scalability of the proposed method across a larger number of worker nodes. Due to hardware constraints in our experimental environment, we conducted logical N-node experiments where multiple worker nodes are simulated on a single physical machine. The logical N-node experiments were designed with 10 logical worker nodes, where each worker node is assigned to process data from either gNB1 or gNB2 without overlap. The 5G-NIDD dataset's gNB1 and gNB2 data are evenly and non-overlappingly distributed across the logical worker nodes, maintaining the same data distribution characteristics as the physical 2-node distributed learning setup. This logical node assignment allows us to evaluate the distributed learning framework's behavior under various scenarios including node failures and data loss. The master node configuration remains the same as in the 2-node distributed learning experiments, handling the aggregation of encoder parameters and training the meta-classifier. Note that these are logical validations rather than physical multi-MEC deployments, which represents a limitation of this study.

5.1.2. Dataset Description

The dataset used in the experiments is 5G-NIDD [33]. This dataset was collected in a real 5G network environment at the University of Oulu, Finland. It was gathered in a 5G setup comprising two base stations and contains normal traffic from real users as well as five types of DoS attacks targeting specific MEC nodes within the 5G environment and three types of port-scan attacks.

The 5G-NIDD dataset provides separated normal and attack traffic collected from each of the two base stations. The proportions of normal traffic, attack traffic, and specific attack types collected from each base station are shown in Figure 6.

Looking at the overall distribution of normal and malicious traffic, gNB1 contained a larger portion of normal traffic at 85.18%, which was 35.18% higher than that of gNB2. In contrast, gNB2 had a larger share of malicious traffic, accounting for 56.46%. When examining the detailed attack traffic, the amounts of SYN flood and UDP flood attacks were about 2.09 times and 1.60 times greater in gNB2, respectively. For the remaining attack types, the differences ranged from approximately 1.0005 to 1.1767 times, indicating that gNB1 and gNB2 had similar volumes of data.

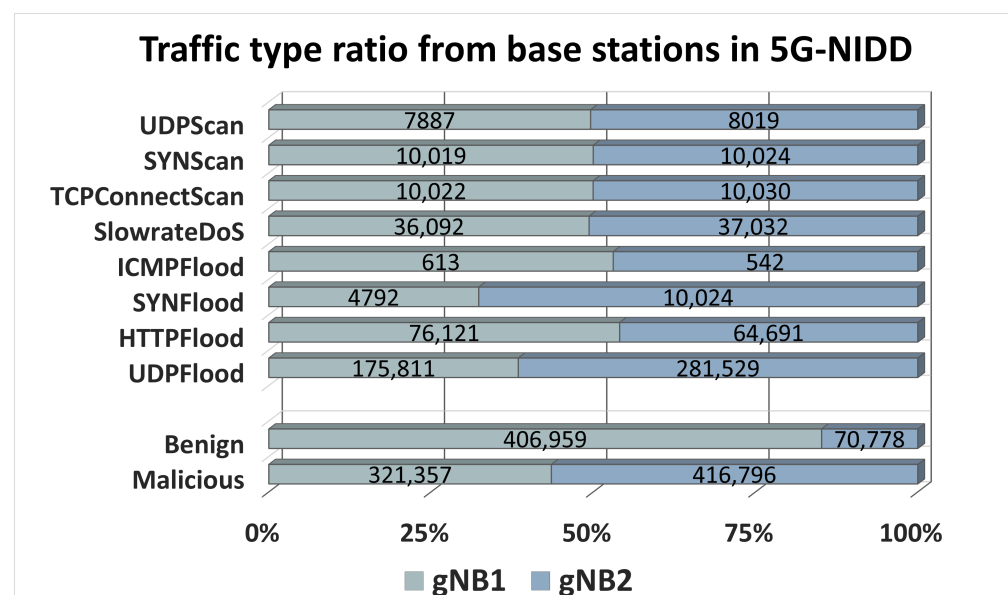


Figure 6. Traffic type ratio from each base station.

5.1.3. Dataset Splitting

In both experiments, the datasets from gNB1 and gNB2 were divided into three subsets: train, validation, and test. First, the original data was split into train + validation and test sets at a ratio of 7:3. Then, the train + validation set was further divided into train and validation sets using the same 7:3 ratio. Following this procedure, the resulting data volumes for gNB1 and gNB2 are shown in Table 2.

Table 2. Traffic distribution of 5G-NIDD dataset across base stations and splits.

Dataset	Benign	HTTP Flood	ICMP Flood	Port Scan	SlowRate DoS	TCP Flood	TCPPort Scan	TCPConnect Scan	UDP Flood
gNB1	406,959	76,121	613	4792	10,019	36,092	10,022	175,811	7887
gNB2	70,778	64,691	542	4929	10,024	37,032	10,030	281,529	8019
gNB1_train	199,409	37,299	300	2349	4909	17,685	4911	86,147	3865
gNB1_val	85,462	15,986	129	1006	2104	7579	2104	36,921	1656
gNB1_test	122,088	22,836	184	1437	3006	10,828	3007	52,743	2366
gNB2_train	34,681	31,699	265	2415	4912	18,145	4915	137,949	3929
gNB2_val	14,864	13,585	114	1035	2105	7777	2106	59,121	1684
gNB2_test	21,233	19,407	163	1479	3007	11,110	3009	84,459	2406

5.1.4. Model Layer Architecture and Hyper-Parameters

The stacking ensemble proposed in this study is composed of autoencoder encoders as the base models and an XGBoost classifier as the meta-classifier. The autoencoder encoders are based on fully connected layers, and the structures and hyper-parameters of each component are summarized in Tables 3 and 4.

Table 3. Autoencoder (base model) architecture and hyper-parameters.

Hyper-Parameter	Configuration
Input dimension	d_{in} (feature size)
Encoder layers	Linear($d_{in} \rightarrow 16$), ReLU, Linear($16 \rightarrow 12$)
Latent dimension (z)	7
Decoder layers	Linear($7 \rightarrow 12$), ReLU, Linear($12 \rightarrow 16$), ReLU, Linear($16 \rightarrow d_{in}$)
Loss function	Mean Squared Error (MSE)

Table 4. XGBoost meta-classifier architecture and hyper-parameters.

Hyper-Parameter	Configuration
Input dimension	Concatenated latent vectors ($7 \times K$)
Classifier type	XGBoost (Gradient Boosting Decision Tree)
Number of estimators	400
Learning rate	0.05
Maximum depth	3
Minimum child weight	5
Gamma (regularization)	0.1
Subsample ratio	0.8
Column sample by tree	0.7
L1 regularization (α)	0.1
L2 regularization (λ)	3.0
Objective function	binary:logistic

All training hyperparameters and stopping rules used in our experiments are detailed below. For base autoencoders, we use Adam optimizer with a learning rate of 0.005 and

MSE loss. The batch size is fixed to 256 for all train/validation/test loaders, and the maximum training epoch is 20 for base models. Early stopping is consistently applied in min mode with a patience of three epochs based on validation loss. For the meta-classifier, we use XGBoost with the hyperparameters specified in Table 4. The XGBoost model uses its built-in early stopping mechanism based on validation error. To ensure reproducibility, we apply random seed control with a fixed seed value of 42 throughout the training pipeline. The operating threshold is selected on the validation set by maximizing validation F1-score, and then fixed for test evaluation.

5.1.5. Evaluation Metrics

This study employs the following metrics to evaluate performance from two perspectives:

- Accuracy;
- Precision;
- Recall;
- F1-score;
- Receiver Operating Characteristic (ROC) curve and Area Under the Curve (AUC).

In the experiments conducted in this study, the AI model performs binary classification of traffic (0: normal, 1: attack). The prediction results of the AI model can be represented using a confusion matrix. The confusion matrix consists of four components: true positive (TP), false positive (FP), false negative (FN), and true negative (TN). These values are used to calculate accuracy, precision, recall, and F1-score.

Accuracy represents the proportion of correct predictions among all predictions made by the model. However, accuracy can be misleading when the test dataset is imbalanced across classes. For example, if the proportion of normal traffic is significantly higher, the model may predict all instances as normal and still achieve a high accuracy value, while performing poorly on minority classes. Therefore, accuracy should only be used as a supplementary metric.

Precision indicates the proportion of correctly predicted positive samples among all samples predicted as positive. This metric is affected by FP, and precision increases as FP decreases. From a security perspective, a low precision value means a high number of false alarms, where normal traffic is misclassified as attacks, thereby increasing the workload for administrators.

Recall indicates the proportion of actual positive samples that are correctly identified as positive. This metric is affected by FN, and recall increases as FN decreases. From a security perspective, a low recall value means that more attacks are missed, which increases the risk of security incidents.

F1-score is the harmonic mean of precision and recall. Since it reflects the balance between FP (affecting precision) and FN (affecting recall), it is suitable as a comprehensive performance metric for evaluating the model.

The ROC curve visualizes the relationship between the true positive rate (TPR, equivalent to recall) and the false positive rate (FPR) at various thresholds. AUC represents the area under the ROC curve. A higher AUC value indicates that the model is more reliable in classifying attacks across different thresholds. The operating threshold τ used in Tables 5 is the decision boundary applied to the meta-classifier's sigmoid output (probability score). We select τ on the validation set by sweeping $[0, 1]$ and maximizing F1-score, and then fix it for test evaluation. Smaller thresholds in the centralized and distributed settings reflect different score distributions and class ratios after feature concatenation; in this paper, τ is defined as the meta-classifier model's decision boundary.

Table 5. Performance comparison between MEC1 and MEC2.

Score	MEC1 (Balanced)	MEC2 (Unbalanced)	
	F1-opt	F1-opt	BalAcc-opt
Best Threshold	0.40	0.66	0.70
Accuracy	0.4449	0.8568	0.7531
Precision	0.4426	0.8592	0.8920
Recall	0.9963	0.9954	0.8092
F1-score	0.6130	0.9223	0.8486
AUC	0.6163		0.6214
Number of Training Samples	356874		238910
Training Time (s)	262.7		177.0

5.2. Experiment 1: Local Single-Node Model Training

The first experiment trains and evaluates the stacking ensemble model at MEC1 and MEC2 using only the data generated from the gNB1 and gNB2 base stations, respectively.

To further characterize this locality-preserving setting, the pre-split class distributions differ substantially across regions: MEC1 (gNB1) is relatively balanced in binary labels (Benign 55.88%, Attack 44.12%), whereas MEC2 (gNB2) is attack-dominant (Benign 14.52%, Attack 85.48%). In addition, the dominant attack composition also varies by node (e.g., UDPFlood-heavy distribution in MEC2). These results confirm regional data heterogeneity across gNB/MEC sites. Therefore, Experiment 1 intentionally preserves regional heterogeneity and should be interpreted as a realistic deployment baseline.

The dataset used for training and evaluation at MEC1 (gNB1 data) contained the following number of samples:

- Train set: 356,874.
- Validation set: 152,947.
- Test set: 218,495.

The dataset used for training and evaluation at MEC2 (gNB2 data) contained the following number of samples:

- Train set: 238,910.
- Validation set: 102,391.
- Test set: 146,273.

Through this experiment, the performance of the model trained on traffic from each independent base station network at a local single node is evaluated.

This experiment presents the results of training autoencoders separately in the MEC1 (gNB1 data) and MEC2 (gNB2 data) environments, and subsequently using the trained encoders as base models for meta-classifier training.

In the MEC1 environment, the losses and validation losses of the three autoencoders converged stably, and the training of all base models took approximately 1 min and 18.99 s. Then, training and validation meta data were generated through the encoder outputs. Based on this meta data, the meta-classifier was trained for about 20.2 s over four epochs. The loss started at 0.6449 and remained in the range of 0.6761–0.6909.

In the MEC2 environment, the three autoencoders completed training in a comparatively shorter time of about 57.80 s, with both training and validation losses converging at low levels. Using the training and validation meta data generated from the encoder outputs, the meta-classifier was trained for about 19.5 s over six epochs. The loss gradually decreased from an initial value of 0.3266 to 0.2411 at the final epoch.

To mitigate class imbalance in MEC2 without synthetic data generation, we applied benign-weighted BCE loss only during meta-classifier training on the train split, while keeping validation and test distributions unchanged.

In both environments, the base models converged stably. For meta-classifier training based on the generated meta data, the MEC1 results showed that the loss remained within a stable range, while the MEC2 results exhibited a gradual decrease in the loss.

In all main experiments, the operating threshold is selected on the validation set by maximizing F1-score (F1-opt) for consistency. For MEC2 in Experiment 1, we additionally report a balanced-accuracy-optimized threshold (BalAcc-opt) as a sensitivity analysis under severe class imbalance.

As shown in Table 5, using the F1-opt operating point, MEC1 adopts a best threshold of 0.40, while MEC2 uses 0.66. Under this setting, MEC1 achieves Accuracy 0.4449, Precision 0.4426, Recall 0.9963, and F1-score 0.6130, whereas MEC2 attains Accuracy 0.8568, Precision 0.8592, Recall 0.9954, and F1-score 0.9223. The AUC is 0.6163 for MEC1 and 0.6214 for MEC2. The number of training samples is 356,874 for MEC1 and 238,910 for MEC2, and the total runtime is 262.7 s for MEC1 and 177.0 s for MEC2.

These results indicate that MEC2 provides higher F1-score and overall balanced performance than MEC1 under the common F1-opt protocol, while MEC1 still exhibits slightly lower AUC.

5.3. Experiment 2: Centralized Model Training

In the second experiment, the traffic data collected from the two base stations was preprocessed at MEC1 and MEC2 through scaling and missing value handling, then converted into a pickle file as a preprocessed dataset and transmitted to MEC3. Based on this data, MEC3 carried out the training of the centralized stacking ensemble model.

The dataset used for training and evaluation at MEC3 (gNB1 data + gNB2 data) contained the following number of samples:

- Train set: 595,784.
- Validation set: 255,338.
- Test set: 364,768.

The model achieved overall high detection performance.

Looking at the detailed performance metrics, the Accuracy, Precision, Recall, and F1-score reached 0.9992, 0.9994, 0.9992, and 0.9993, respectively, while the AUC was 0.9999. The best threshold for the meta-classifier was 0.70.

The time metrics refer to the working time of each MEC node. MEC1 (worker1) and MEC2 (worker2) required 00:27 and 00:13, respectively, to preprocess and transmit the data to the Master (MEC3). The Master (MEC3) then required 05:35 to train the stacking ensemble model using the received data. Since the worker nodes operate in parallel, the total execution time was approximately 06:02 ($\max(00:27, 00:13) + 05:35$), which reflects the characteristics of the centralized learning architecture, where all data must be aggregated at the Master node (Table 6).

In summary, centralized learning achieved stable detection performance with high accuracy, precision, and recall, but it showed the limitation of concentrated workload on the Master node and relatively longer working time during the training process.

Table 6. Working time of centralized learning (Min:Sec).

	Worker1	Worker2	Master	Total Time (Parallel)
Time	00:27	00:13	05:35	06:02

5.4. Experiment 3: Multi-Edge Distributed Model Training

The third experiment begins with preprocessing data generated from two base stations on each worker node (MEC1, MEC2), followed by training the base model for the stacking ensemble model. Once base model training is complete, the worker nodes extract the encoder parameters from the trained autoencoder and transmit them to the master node, MEC3, along with the preprocessed validation and test sets. The master node reconstructs the base model using the received base model parameters to train the encoder. It then trains and evaluates the meta-classifier model based on the received preprocessed validation and test sets. The composition of the dataset used in this experiment is identical to that in Experiment 2.

In the distributed learning environment, the training convergence was observed across both worker nodes. Each worker's three autoencoders converged to stable reconstruction losses within 3–5 epochs, similar to the local single-node experiments. As shown in Figure 7, the master node's XGBoost meta-classifier training showed efficient convergence with validation error steadily decreasing. The distributed training maintained stable learning dynamics despite the increased model complexity from combining encoders from multiple nodes. The model exhibited very high detection performance.

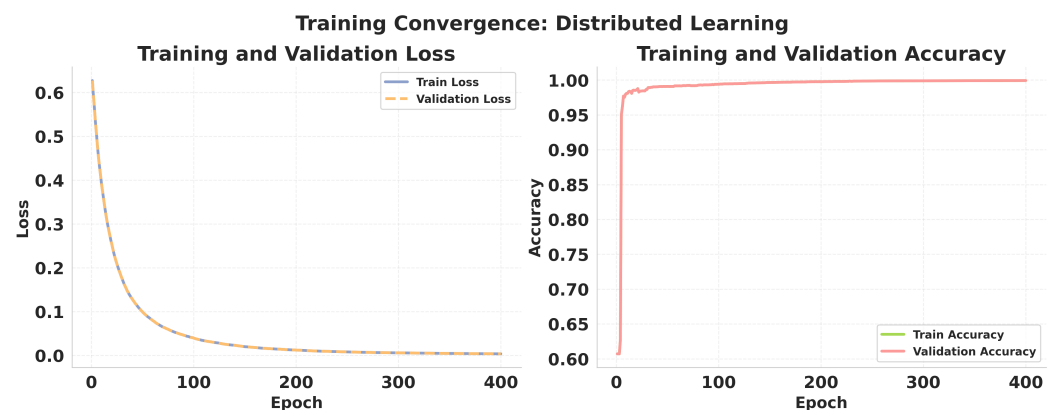


Figure 7. Training convergence of the distributed learning model showing training and validation loss (**left**) and accuracy (**right**) over 400 epochs.

Looking at the detailed performance metrics, the Accuracy, Precision, Recall, and F1-score were 0.9993, 0.9996, 0.9992, and 0.9994, respectively, again with an AUC of 0.9999. The best threshold for the meta-classifier was 0.75.

The time metrics refer to the working time measured at each MEC node. MEC1 (worker1) and MEC2 (worker2) required 04:29 and 03:07, respectively, to preprocess the data and train the base models, and then to transmit the meta-training data and trained base model parameters to the Master node (MEC3). The Master (MEC3) required an additional 00:04 to load the received base models, reconstruct the stacking ensemble model, and perform training. Since the worker nodes operate in parallel, the total execution time was approximately 04:33 (max(04:29, 03:07) + 00:04) (Table 7).

In summary, distributed learning maintained both precision and recall at high levels, achieving performance comparable to centralized learning in F1-score and AUC. Furthermore, the total working time was shorter than that of centralized learning, demonstrating improved efficiency while maintaining comparable performance.

Table 7. Working time of distributed learning (Min:Sec).

	Worker1	Worker2	Master	Total Time (Parallel)
Time	04:29	03:07	00:04	04:33

5.5. Experiment 4: Logical N-Node Distributed Model Training

The fourth experiment extends the distributed learning framework to evaluate scalability and robustness across multiple worker nodes. This experiment was designed to assess how the proposed method performs when the number of worker nodes increases, and how it handles realistic distributed system challenges such as node failures and data loss.

The logical N-node experiment configuration consists of 10 logical worker nodes, where each worker node processes data from either gNB1 or gNB2 without overlap. The 5G-NIDD dataset's gNB1 and gNB2 data are evenly distributed across the logical worker nodes, maintaining the same underlying data characteristics as the two-node distributed learning physical setup. The master node aggregates encoder parameters from all active worker nodes and trains the XGBoost meta-classifier using the concatenated latent features.

We conducted multiple experimental scenarios to evaluate different aspects of the distributed learning framework:

1. **Baseline scenario:** All 10 worker nodes are active with no failures or data loss. This scenario establishes the baseline performance when all nodes contribute their encoder parameters to the meta-classifier training.
2. **Node failure scenarios:** We simulated node failures at different rates (10%, 20%, 30%, and 40% of worker nodes) to evaluate how the system responds when some nodes become unavailable. When a node fails, its encoder parameters are not available for meta-classifier training, reducing the input dimension to the meta-classifier accordingly.
3. **Data loss scenarios:** We simulated data loss at different rates (10%, 20%, 30%, and 40% of total samples) to evaluate how the system performs when training data is partially unavailable. Data loss is distributed proportionally across nodes and data splits (train, validation, test).

For each scenario, the experimental procedure follows the same workflow as described in Experiment 3, but with the extended number of worker nodes. Each worker node trains three autoencoder encoders locally using its assigned data subset, extracts the encoder parameters, and transmits them to the master node along with preprocessed validation and test sets. The master node reconstructs all available encoders, creates the meta-dataset by concatenating latent features from all active nodes, and trains the XGBoost meta-classifier.

The dataset composition for logical N-node experiments follows the same splitting strategy as Experiments 2 and 3. Each worker node processes data from either gNB1 or gNB2, maintaining the same train/validation/test ratio of 7:3. The total number of training samples across all worker nodes is the same as in the centralized and two-node distributed experiments, but distributed across 10 logical nodes instead of two physical nodes.

The performance evaluation for logical N-node experiments includes the same metrics as previous experiments: Accuracy, Precision, Recall, F1-score, and AUC. Additionally, we analyze the impact of node failures and data loss on detection performance and communication overhead. Table 8 summarizes the performance metrics across different scenarios. In the baseline logical N-node scenario with all 10 workers active, the meta-classifier achieved Accuracy 0.9995, Precision 0.9997, Recall 0.9994, F1-score 0.9996, and AUC 0.9999, with a total training time of 719 s at the master side.

These results confirm that the logical N-node distributed learning framework maintains near-perfect detection performance while scaling to a larger number of logical worker nodes, even under node failure and data loss conditions.

Table 8. Logical N-node distributed learning performance under different scenarios.

Scenario	Accuracy	Precision	Recall	F1-Score	AUC
Baseline (10 nodes, no failure, no data loss)	0.9995	0.9997	0.9994	0.9996	0.9999
Node Failure 10%	0.9995	0.9997	0.9994	0.9996	0.9999
Node Failure 20%	0.9995	0.9998	0.9994	0.9996	0.9999
Node Failure 30%	0.9995	0.9998	0.9995	0.9996	0.9999
Node Failure 40%	0.9994	0.9997	0.9994	0.9995	0.9999
Data Loss 10%	0.9995	0.9997	0.9994	0.9996	0.9999
Data Loss 20%	0.9995	0.9995	0.9996	0.9996	0.9999
Data Loss 30%	0.9995	0.9997	0.9995	0.9996	0.9999
Data Loss 40%	0.9996	0.9998	0.9995	0.9997	0.9999

6. Analysis of Experimental Results

6.1. Performance Comparison of Local Models in Experiment 1

Comparing the local models under the primary F1-opt protocol, MEC1 achieved Accuracy 0.4449, Precision 0.4426, Recall 0.9963, and F1-score 0.6130, while MEC2 achieved Accuracy 0.8568, Precision 0.8592, Recall 0.9954, and F1-score 0.9223 (Table 5). Accordingly, MEC2 showed better performance than MEC1 across all metrics, particularly in F1-score, despite both models achieving near-perfect recall (Figure 8a).

For threshold-independent ranking metrics, ROC-AUC was 0.6163 for MEC1 and 0.6214 for MEC2, indicating comparable ranking-based discrimination in this local setting.

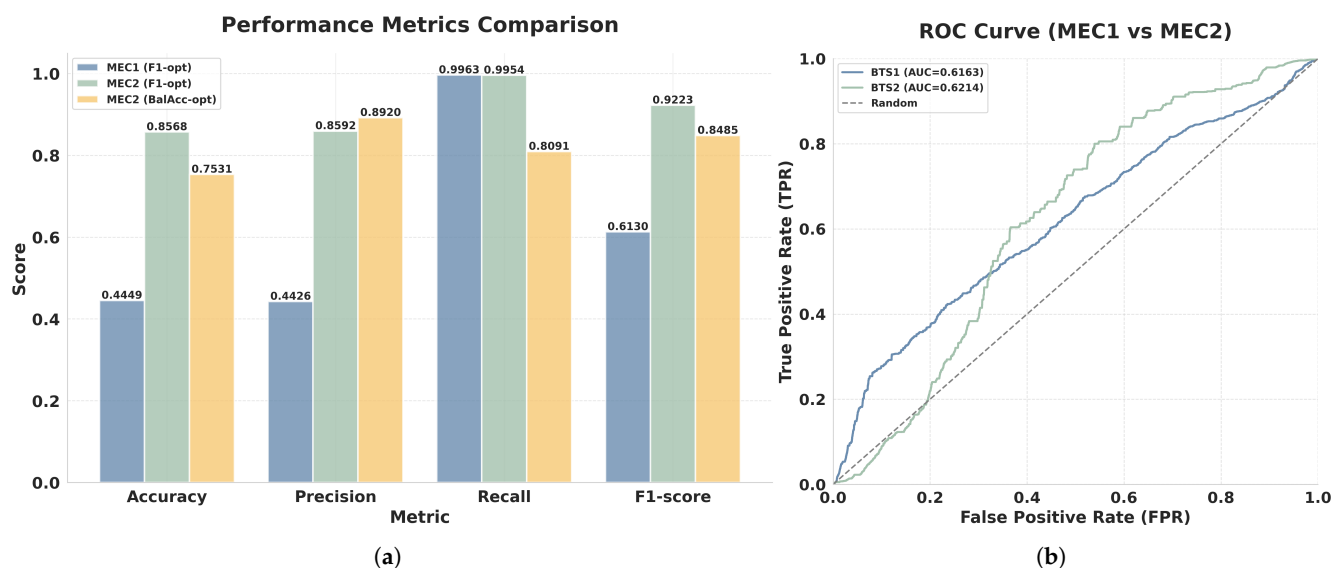


Figure 8. Local single-node performance comparison between MEC1 and MEC2. (a) Local single-node performance comparison between MEC1 and MEC2. (b) ROC curve comparison between the local models trained at MEC1 and MEC2.

From the perspective of training resources, MEC1 used 356,874 training samples and required 262.7 s, while MEC2 used 238,910 samples and required 177.0 s. Thus, MEC2 was faster with fewer samples in this local training setup.

In summary, MEC2 provides higher Accuracy and F1-score under the F1-opt operating point, while both MECs attain very high recall. Therefore, in Experiment 1, the main difference between MEC1 and MEC2 is the overall balance between precision and recall under heterogeneous class distributions with severe class imbalance.

6.2. Performance Comparison of AI Models in Experiments 2 and 3

Centralized and distributed learning were compared in terms of performance, and both methods achieved near-perfect detection metrics (Table 9). Accuracy was 0.9992 in centralized learning and 0.9993 in distributed learning, while Precision remained above 0.9994 in both settings. Recall and F1-score also exceeded 0.9992 for both architectures, and AUC stayed at 0.9999, indicating virtually identical classification stability.

In contrast, the time characteristics differed more clearly. The centralized setting required a total of 06:02, whereas the distributed setting completed in 04:33, reflecting the benefit of parallelized base-model training at worker MECs. As illustrated in Figure 9, these results visually confirm that the proposed distributed learning maintains centralized-level performance while reducing overall training time. The slight performance improvement comes from keeping node-specific features. Each MEC trains its own encoders on local data, and the master concatenates the resulting latent features for stacking. This keeps different traffic patterns and helps the meta-classifier reduce false negatives without increasing false positives; in our experiments, Accuracy, Recall, and F1 showed slight improvements while AUC stayed almost the same.

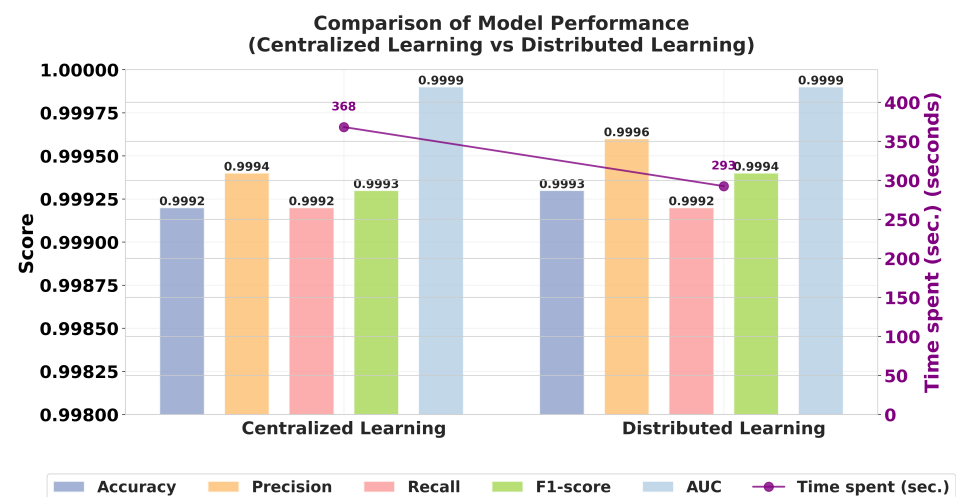


Figure 9. Comparison of model performance between centralized and distributed learning

Following the comparison of performance metrics and training time, the network traffic generated by the two approaches was analyzed. In the centralized learning (Master) scenario, a total of 617.85 MB of data and 46,088 packets were generated, with an average bandwidth of 112.67 Mbps and a maximum bandwidth of 183.46 Mbps.

In contrast, the distributed learning (Master) scenario produced 64.21 MB of data and 4911 packets. This significant reduction, approximately one-tenth of the centralized case, resulted from transmitting binary variable data containing preprocessed features directly usable by the AI model instead of raw traffic data. The average bandwidth was reduced to 53.86 Mbps, less than half of the centralized value, while the maximum bandwidth was similar at 173.98 Mbps. However, the traffic duration was considerably shorter.

Additionally, a key analytical point is that the proposed distributed learning architecture maintained comparable performance while improving efficiency, despite utilizing more encoders than the centralized approach. While centralized learning trained the meta-classifier using three encoders (meta-input dimension of 21), the distributed learning method combined a total of six latent features by utilizing all three encoders trained at MEC1 and MEC2, respectively (meta-input dimension of 42). Despite the input dimension to the meta-classifier doubling from 21 to 42, the distributed approach demonstrated comparable performance with slight gains in some metrics while reducing computational time.

These results indicate that in distributed learning, combining latent features generated across two nodes absorbs data imbalance and distribution differences, improves learning efficiency, and secures more stable integrated performance.

6.3. Performance Comparison Across Experiments

Table 9 summarizes the performance metrics and training time across all experimental scenarios. As shown in the table, local single-node learning showed lower performance (average Accuracy 0.6508, F1-score 0.7677) due to limited data diversity, with notable differences between MEC1 and MEC2 reflecting the heterogeneous traffic distribution across base stations. Centralized and distributed learning both achieved near-perfect detection performance, with distributed learning maintaining centralized-level performance (Accuracy 0.9993 vs. 0.9992, F1-score 0.9994 vs. 0.9993) while reducing training time from 368 s to 293 s. The logical N-node distributed learning demonstrated scalability with the highest performance metrics (Accuracy 0.9995, F1-score 0.9996) even under node failure and data loss conditions, though with increased training time (719 s) due to the larger number of worker nodes.

Table 9. Performance metrics of different learning methods.

Learning Method	Threshold	Accuracy	Precision	Recall	F1-Score	AUC	Time Spent (s)
Local single-node: MEC1	0.40	0.4449	0.4426	0.9963	0.6130	0.6163	262.7
Local single-node: MEC2	0.66	0.8568	0.8592	0.9954	0.9223	0.6214	177.0
Local single-node Learning (avg)	–	0.6508	0.6509	0.9959	0.7677	0.6188	219.9
Centralized Learning	0.70	0.9992	0.9994	0.9992	0.9993	0.9999	368
Distributed Learning (proposed)	0.75	0.9993	0.9996	0.9992	0.9994	0.9999	293
Logical N-Node Distributed Learning (baseline)	0.67	0.9995	0.9997	0.9994	0.9996	0.9999	719

Figure 10 visually compares the performance metrics across all learning methods. The figure clearly shows that local single-node learning exhibits substantially lower performance across all metrics compared to centralized and distributed learning, which both achieve near-perfect scores. Distributed learning shows slight improvements over centralized learning in Accuracy, Precision, and F1-score, while maintaining identical AUC performance. The logical N-node distributed learning achieves the highest performance metrics, demonstrating the scalability of the proposed framework.

Figure 11 compares the total training time across different learning methods. Local single-node learning requires the shortest time (219.9 s on average) but at the cost of significantly lower performance. Centralized learning takes the longest time (368 s) due to the need to aggregate all data at the master node. Distributed learning achieves a balance between performance and efficiency, reducing training time to 293 s while maintaining comparable or slightly improved performance. The logical N-node distributed learning requires 719 s, reflecting the increased computational overhead from processing data across 10 worker nodes, but still demonstrates efficient scalability given the larger number of nodes involved.

6.4. Performance Analysis of Logical N-Node Distributed Learning

The logical N-node distributed learning experiments were conducted to evaluate the scalability and robustness of the proposed framework across a larger number of worker nodes. The baseline logical N-node scenario (10 worker nodes, no failures, no data loss) establishes the reference performance when all nodes contribute their encoder

parameters. In this configuration, the meta-classifier receives latent features from 30 encoders ($10 \text{ nodes} \times 3 \text{ encoders per node}$), resulting in a meta-input dimension of 210 ($30 \text{ encoders} \times 7 \text{ latent dimensions}$). This increased input dimension compared to the two-node setup (42 dimensions) allows the meta-classifier to learn from a more diverse set of feature representations while still achieving Accuracy 0.9995, Precision 0.9997, Recall 0.9994, F1-score 0.9996, and AUC 0.9999.

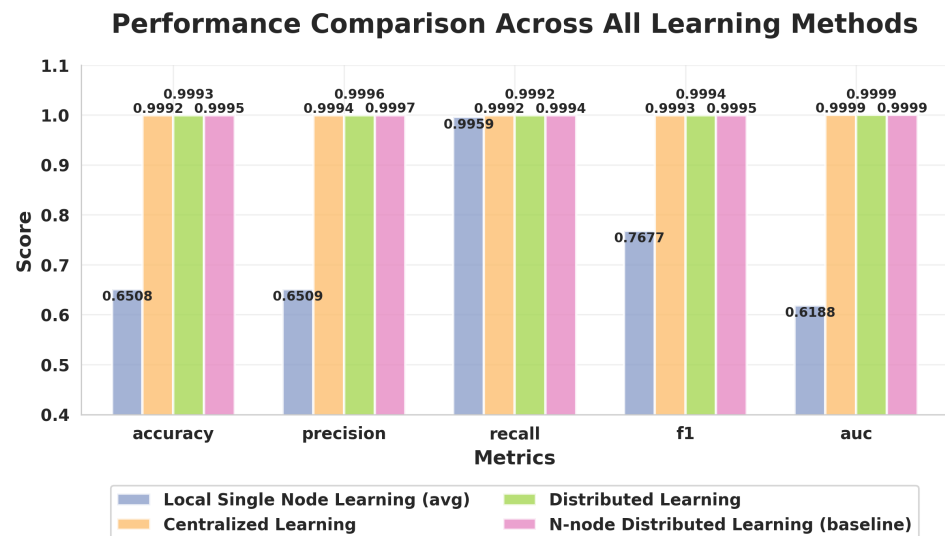


Figure 10. Performance comparison across all learning methods.

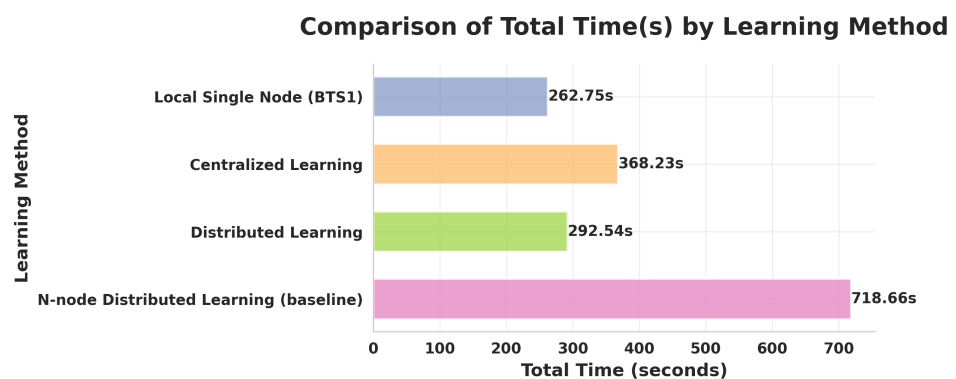


Figure 11. Comparison of total training time across different learning methods.

The node failure scenarios evaluate how the system responds when a portion of worker nodes become unavailable. As the failure rate increases from 10% to 40%, the number of active nodes decreases from 9 to 6, and the meta-input dimension reduces proportionally. As shown in Table 8, detection metrics remain near-perfect across all failure rates, with Accuracy and F1-score staying above 0.9994 even at 40% failure rate. The resulting performance curves in Figure 12 visually confirm this robustness, indicating graceful degradation and resilience against node outages.

The data loss scenarios evaluate the system's performance when training data is partially unavailable. Data loss affects the base model training at worker nodes, potentially reducing the quality of encoder representations. As shown in Table 8, as the data loss rate increases from 10% to 40%, all detection metrics remain near-perfect, with Accuracy and F1-score staying above 0.9995 across all scenarios. Figure 13 visually illustrates that Accuracy, Precision, Recall, F1-score, and AUC all stay near 1.0, with only minor degradation, demonstrating robustness to partial data unavailability.

Overall, the logical N-node distributed learning framework maintains the core advantages observed in the two-node setup: reduced communication overhead compared

to centralized learning, preservation of node-specific traffic patterns, and efficient meta-classifier training. In addition, the extended node configuration confirms that near-perfect detection quality can be sustained even when scaling to 10 logical MEC nodes and under realistic node failure and data loss conditions.

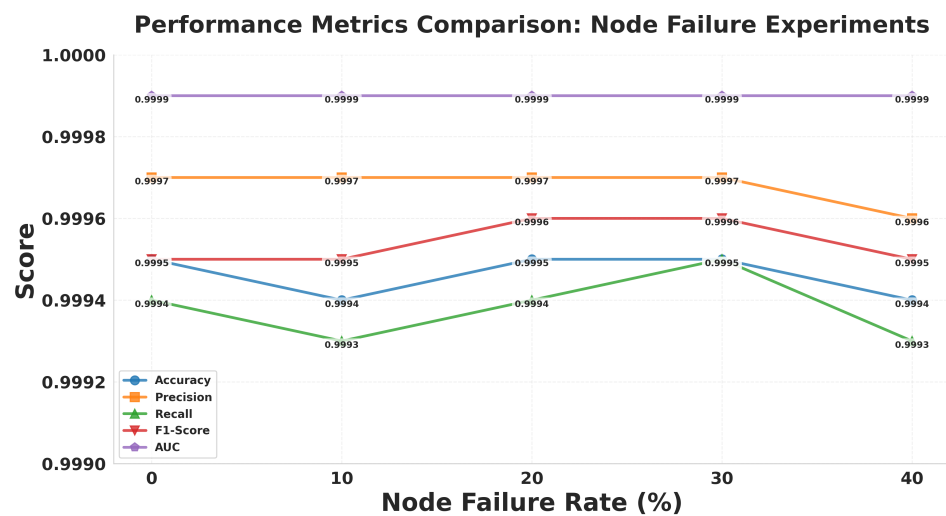


Figure 12. Logical N-node distributed learning performance under different node failure rates.

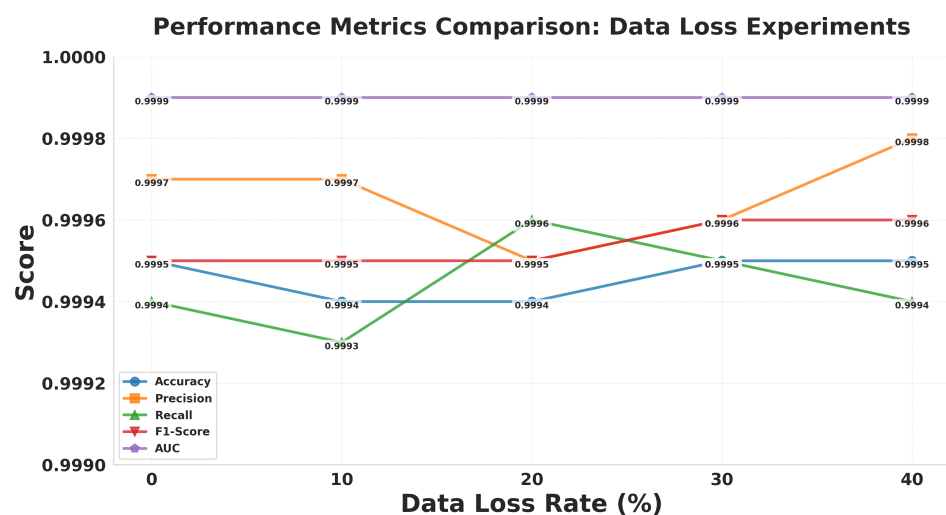


Figure 13. Logical N-node distributed learning performance under different data loss rates.

7. Conclusions

This study designed a stacking ensemble-based intrusion detection model for distributed learning in a multi-MEC environment and validated its performance using the real 5G-NIDD dataset. The proposed architecture enables efficient training and detection without centralizing raw traffic by training base models locally at each worker MEC and training and inference of the meta-classifier at the master MEC.

In the experiments, four experimental scenarios were compared: local single-node learning, centralized learning, distributed learning, and logical N-node distributed learning. The results showed that distributed learning achieved comparable or slightly improved performance compared to centralized learning in terms of accuracy, recall, and F1-score, while also reducing transmission traffic and training time, and maintaining stable performance in AUC. The logical N-node experiments further demonstrated the scalability and robustness of the proposed framework. In addition, performance variation was observed according to the traffic distribution of each base station, clearly demonstrating the necessity and effectiveness of multi-MEC integrated meta learning. This study shows that the distributed

learning approach is an effective alternative for the design of MEC-based security functions in the 6G era. A limitation of this study is that experiments were conducted with two MEC nodes because additional physical MEC nodes were not available in our experimental environment; future work will test larger multi-MEC deployments and measure latency and detection quality as nodes increase. With more worker MECs, transfer volume and training time are expected to increase, while detection quality may benefit from more diverse traffic patterns; we will evaluate this trade-off in a larger testbed.

Author Contributions: Conceptualization, M.-G.K. and J.K.; methodology, M.-G.K. and J.K.; data curation, M.-G.K.; formal analysis, M.-G.K. and J.K.; writing—original draft preparation, M.-G.K.; writing—review and editing, M.-G.K. and J.K. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.RS-2024- 00444170, Research and international collaboration on trust model-based intelligent incident response technologies in 6G open network environment).

Data Availability Statement: The data presented in this study are available in IEEE Data-Port at <https://iee-dataport.org/documents/5g-nidd-comprehensive-network-intrusion-detection-dataset-generated-over-5g-wireless> (accessed on 19 March 2026). These data were derived from public domain resources.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. ITU-R WP5D. *Framework and overall objectives of the future development of IMT for 2030 and beyond*; Recommendation M.2160; ITU: Geneva, Switzerland, 2023.
2. Nithyalakshmi, N.; Muthukumaravel, A. 5G-Optimized Intrusion Detection using Hybrid Data Mining and Deep Learning Model. In *Proceedings of the 2025 International Conference on Knowledge Engineering and Communication Systems (ICKECS)*; IEEE: Piscataway, NJ, USA, 2025; pp. 1–6. <https://doi.org/10.1109/ICKECS65700.2025.11035396>.
3. Ilias, L.; Doukas, G.; Lamprou, V.; Ntanos, C.; Askounis, D. Convolutional Neural Networks and Mixture of Experts for Intrusion Detection in 5G Networks and beyond. *arXiv* **2025**, arXiv:2412.03483. <https://doi.org/10.48550/arXiv.2412.03483>.
4. Yadav, N.; Pande, S.; Khamparia, A.; Gupta, D. Intrusion Detection System on IoT with 5G Network Using Deep Learning. *Wirel. Commun. Mob. Comput.* **2022**, 2022, 9304689. <https://doi.org/10.1155/2022/9304689>.
5. Tu, S.; Waqas, M.; Badshah, A.; Yin, M.; Abbas, G. Network Intrusion Detection System (NIDS) Based on Pseudo-Siamese Stacked Autoencoders in Fog Computing. *IEEE Trans. Serv. Comput.* **2023**, 16, 4317–4327. <https://doi.org/10.1109/TSC.2023.3319953>.
6. Djaidja, T.E.T.; Brik, B.; Mohammed Senouci, S.; Boualouache, A.; Ghamri-Doudane, Y. Early Network Intrusion Detection Enabled by Attention Mechanisms and RNNs. *IEEE Trans. Inf. Forensics Secur.* **2024**, 19, 7783–7793. <https://doi.org/10.1109/TIFS.2024.3441862>.
7. Chen, M.; Gunduz, D.; Huang, K.; Saad, W.; Bennis, M.; Feljan, A.V.; Poor, H.V. Distributed Learning in Wireless Networks: Recent Progress and Future Challenges. *IEEE J. Sel. Areas Commun.* **2021**, 39, 3579–3605. <https://doi.org/10.1109/jsac.2021.3118346>.
8. Cao, X.; Başar, T.; Diggavi, S.; Eldar, Y.C.; Letaief, K.B.; Poor, H.V.; Zhang, J. Communication-efficient distributed learning: An overview. *IEEE J. Sel. Areas Commun.* **2023**, 41, 851–873.
9. Yun, J.; Goh, Y.; Yoo, W.; Chung, J.M. 5G Multi-RAT URLLC and eMBB Dynamic Task Offloading With MEC Resource Allocation Using Distributed Deep Reinforcement Learning. *IEEE Internet Things J.* **2022**, 9, 20733–20749. <https://doi.org/10.1109/JIOT.2022.3177425>.
10. Zhang, F.; Han, G.; Liu, L.; Jiang, J.; Li, A.; Zhu, S. Secure Data Offloading and Resource Allocation Against Hybrid Intrusions for IIoT: A Fully Decentralized Framework. *IEEE Internet Things J.* **2025**, 12, 9218–9237. <https://doi.org/10.1109/JIOT.2024.3506595>.
11. Liu, X.; Chen, A.; Zheng, K.; Chi, K.; Yang, B.; Taleb, T. Distributed computation offloading for energy provision minimization in WP-MEC networks with multiple HAPs. *IEEE Trans. Mob. Comput.* **2024**, 24, 2673–2689.
12. Cheng, S.M.; Hong, B.K.; Hung, C.F. Attack Detection and Mitigation in MEC-Enabled 5G Networks for AIoT. *IEEE Internet Things Mag.* **2022**, 5, 76–81. <https://doi.org/10.1109/IOTM.001.2100144>.
13. Man, D.; Zeng, F.; Yang, W.; Yu, M.; Lv, J.; Wang, Y. Intelligent Intrusion Detection Based on Federated Learning for Edge-Assisted Internet of Things. *Secur. Commun. Netw.* **2021**, 2021, 9361348. <https://doi.org/10.1155/2021/9361348>.

14. Uddin, M.A.; Chu, N.H.; Rafeh, R.; Barika, M. A Scalable Hierarchical Intrusion Detection System for Internet of Vehicles. *IEEE Internet Things J.* **2025**, *12*, 41591–41607. <https://doi.org/10.1109/jiot.2025.3590966>.
15. Lyu, S.; Lin, Z.; Qu, G.; Chen, X.; Huang, X.; Li, P. Optimal resource allocation for u-shaped parallel split learning. In *Proceedings of the 2023 IEEE Globecom Workshops (GC Wkshps)*; IEEE: Piscataway, NJ, USA, 2023; pp. 197–202.
16. Oh, Y.; Lee, J.; Brinton, C.G.; Jeon, Y.S. Communication-efficient split learning via adaptive feature-wise compression. *IEEE Trans. Neural Netw. Learn. Syst.* **2025**, *36*, 10844–10858.
17. Sáez-de Cámara, X.; Flores, J.L.; Arellano, C.; Urbieto, A.; Zurutuza, U. Clustered federated learning architecture for network anomaly detection in large scale heterogeneous IoT networks. *Comput. Secur.* **2023**, *131*, 103299.
18. Adjewa, F.; Esseghir, M.; Merghem-Boulahia, L. Efficient federated intrusion detection in 5g ecosystem using optimized bert-based model. In *Proceedings of the 2024 20th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*; IEEE: Piscataway, NJ, USA, 2024; pp. 62–67.
19. He, N.; Zhang, Z.; Wang, X.; Gao, T. Efficient Privacy-Preserving Federated Deep Learning for Network Intrusion of Industrial IoT. *Int. J. Intell. Syst.* **2023**, *2023*, 2956990.
20. Shen, J.; Yang, W.; Chu, Z.; Fan, J.; Niyato, D.; Lam, K.Y. Effective intrusion detection in heterogeneous Internet-of-Things networks via ensemble knowledge distillation-based federated learning. In *Proceedings of the ICC 2024-IEEE International Conference on Communications*; IEEE: Piscataway, NJ, USA, 2024; pp. 2034–2039.
21. Zang, M.; Zheng, C.; Koziak, T.; Zilberman, N.; Dittmann, L. Federated Learning-Based In-Network Traffic Analysis on IoT Edge. In *Proceedings of the 2023 IFIP Networking Conference (IFIP Networking)*; IEEE: Piscataway, NJ, USA, 2023; pp. 1–6. <https://doi.org/10.23919/IFIPNetworking57963.2023.10186438>.
22. de Carvalho Bertoli, G.; Junior, L.A.P.; Saotome, O.; Dos Santos, A.L. Generalizing intrusion detection for heterogeneous networks: A stacked-unsupervised federated learning approach. *Comput. Secur.* **2023**, *127*, 103106.
23. Nakip, M.; Gül, B.C.; Gelenbe, E. Decentralized online federated g-network learning for lightweight intrusion detection. In *Proceedings of the 2023 31st International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*; IEEE: Piscataway, NJ, USA, 2023; pp. 1–8.
24. Santhadevi, D.; Janet, B. Stacked deep learning framework for edge-based intelligent threat detection in IoT network. *J. Supercomput.* **2023**, *79*, 12622–12655.
25. Cui, J.; Zong, L.; Xie, J.; Tang, M. A novel multi-module integrated intrusion detection system for high-dimensional imbalanced data. *Appl. Intell.* **2023**, *53*, 272–288.
26. Fathima, N.; Pramod, A.; Srivastava, Y.; Thomas, A.M.; Syed Ibrahim, S.P.; Chandran, K.R. Two-stage deep stacked autoencoder with shallow learning for network intrusion detection system. *arXiv* **2021**, arXiv:2112.03704.
27. Abbas, Q.; Hina, S.; Sajjad, H.; Zaidi, K.S.; Akbar, R. Optimization of predictive performance of intrusion detection system using hybrid ensemble model for secure systems. *PeerJ Comput. Sci.* **2023**, *9*, e1552.
28. Yin, X.; Fang, W.; Liu, Z.; Liu, D. A novel multi-scale CNN and Bi-LSTM arbitration dense network model for low-rate DDoS attack detection. *Sci. Rep.* **2024**, *14*, 5111.
29. Ali, M.; Haque, M.u.; Durad, M.H.; Usman, A.; Mohsin, S.M.; Mujlid, H.; Maple, C. Effective network intrusion detection using stacking-based ensemble approach. *Int. J. Inf. Secur.* **2023**, *22*, 1781–1798. <https://doi.org/10.1007/s10207-023-00718-7>.
30. Rais, R.N.B.; Khalid, O.; Nazar, J.e.; Khan, M.U.S. Analysis of Intrusion Detection Using Ensemble Stacking-Based Machine Learning Techniques in IoT Networks. In *Proceedings of the 2023 International Conference on Advances in Computing Research (ACR'23)*; Daimi, K., Al Sadoon, A., Eds.; Springer: Cham, Switzerland, 2023; pp. 329–344.
31. nvidia. CUDA and cuDNN Images from gitlab.com/nvidia/cuda. Available online: <https://hub.docker.com/r/nvidia/cuda> (accessed on 19 March 2026).
32. kaazing. Capture network traffic in Docker or Docker Compose containers using tcpdump for Wireshark analysis. Available online: <https://hub.docker.com/r/kaazing/tcpdump> (accessed on 19 March 2026).
33. Siriwardhana, Y.; Samarakoon, S.; Porambage, P.; Liyanage, M.; Chang, S.Y.; Kim, J.; Kim, J.; Ylianttila, M. Descriptor: 5G Wireless Network Intrusion Detection Dataset (5G-NIDD). *IEEE Data Descr.* **2025**, *2*, 358–369. <https://doi.org/10.1109/IEEEDATA.2025.3592888>.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.